

Git & Github 정리

Git & Github

프로젝트 버전 관리 시스템

* Git

* Github

*Git 사용법

*깃의 세 가지 관리 영역

*gitignore

*커밋 이력

*실수 했을 때

*브랜치 관리하기

**깃허브!

Git & Github

프로젝트 버전 관리 시스템

* Git

1. 버전 관리 : 프로젝트 내 문서의 수정 이력을 체계적으로 관리할 수 있다. 문서 내용을 특정 시점으로 되돌리거나, 서비스 버전과 개발 버전을 별도로 운영하는 등 프로젝트 관리에 대해 다양한 편의를 제공한다.

2. 협업 체계 : 여러 사람이 하나의 프로젝트를 수행할 때, 하나의 저장소를 기반으로 자신만의 개발 버전을 관리할 수 있어 도움이 된다.

* Github

: 깃 저장소 호스팅을 지원하는 웹 서비스이다. 더 편리하게 깃 저장소를 관리할 수 있는 플랫폼이다.

*Git 사용법

: 커맨드 라인 인터페이스(Command-Line Interface, CLI)가 가장 많이 사용된다.

사용할 폴더 생성 - Git Bash Here 클릭 - 아래 명령어들로 폴더 초기화. 깃 사용 준비

- **Git Bash** : 사용자가 깃 명령어를 실행할 수 있게 해주는 명령행 인터페이스 프로그램

```
$ git config --global user.name "사용자명 (닉네임)"
```

- 깃 저장소에서 사용할 사용자 정보 중 사용자명을 설정하는 깃 명령어

```
$ git config --global user.email "사용자메일"
```

- 깃 저장소에서 사용할 사용자 정보 중 사용자 메일 주소를 설정하는 깃 명령어

```
$ git init
```

- 터미널이 열려 있는 폴더를 깃 저장소로 초기화하는 깃 명령어

```
$ ls
```

- 터미널이 열려 있는 폴더 내에 있는 항목들을 나열하는 시스템 명령어

```
ls -al
```

- 터미널이 열려 있는 폴더 내에 있는 숨긴 항목까지 나열하는 시스템 명령어

*깃의 세 가지 관리 영역

Working Directory → Staging Area → Repository

- **Working Directory** : 개발자가 실제로 관리 및 작업하는 영역
- **Staging Area** : 수정 이력을 기록할 파일을 대기 시키는 장소, 가상의 깃 관리 영역
- **Repository** : 대기 중이던 파일들의 수정 이력이 최종적으로 기록(커밋, commit)되는 장소, 가상의 깃 관리 영역

```
$ git status
```

- 깃 프로젝트 상태를 확인하는 깃 명령어

```
$ git add
```

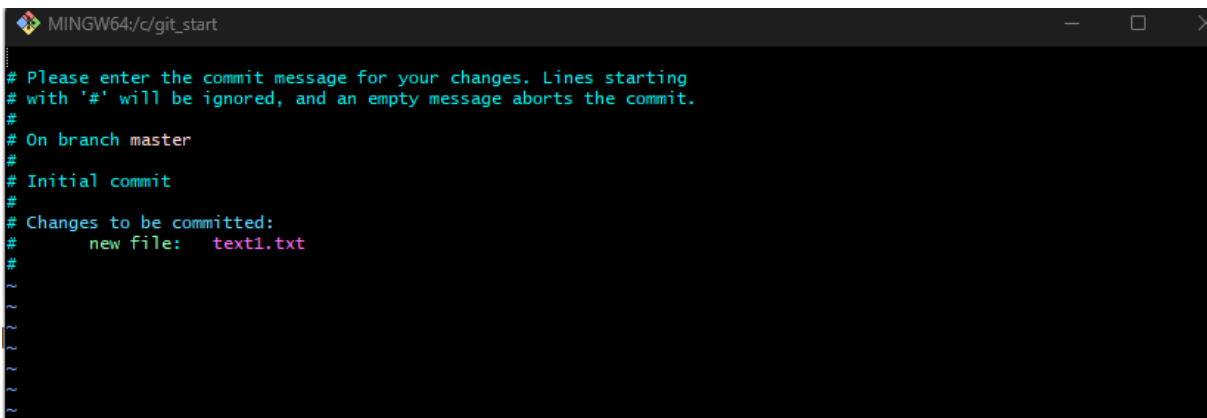
- 워킹 디렉토리 내 문서를 스테이징 영역에 추가하는 깃 명령어

```
$ git commit
```

- 스테이징 영역 내에 대기 중인 문서를 리포지토리에 추가하는 깃 명령어

```
$ git log
```

- 커밋한 수정 이력을 확인하는 깃 명령어



```

MINGW64:/c/git_start
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
#
# Initial commit
#
# Changes to be committed:
#   new file:   text1.txt
#
~
~
~
~
~
~

```

VI / vim (커밋 메시지 편집 화면)

커밋 이후 달라진 터미널의 인터페이스를 VI 또는 vim이라 부른다. 터미널 에디터로서 커밋 메시지를 수정하기 위한 인터페이스

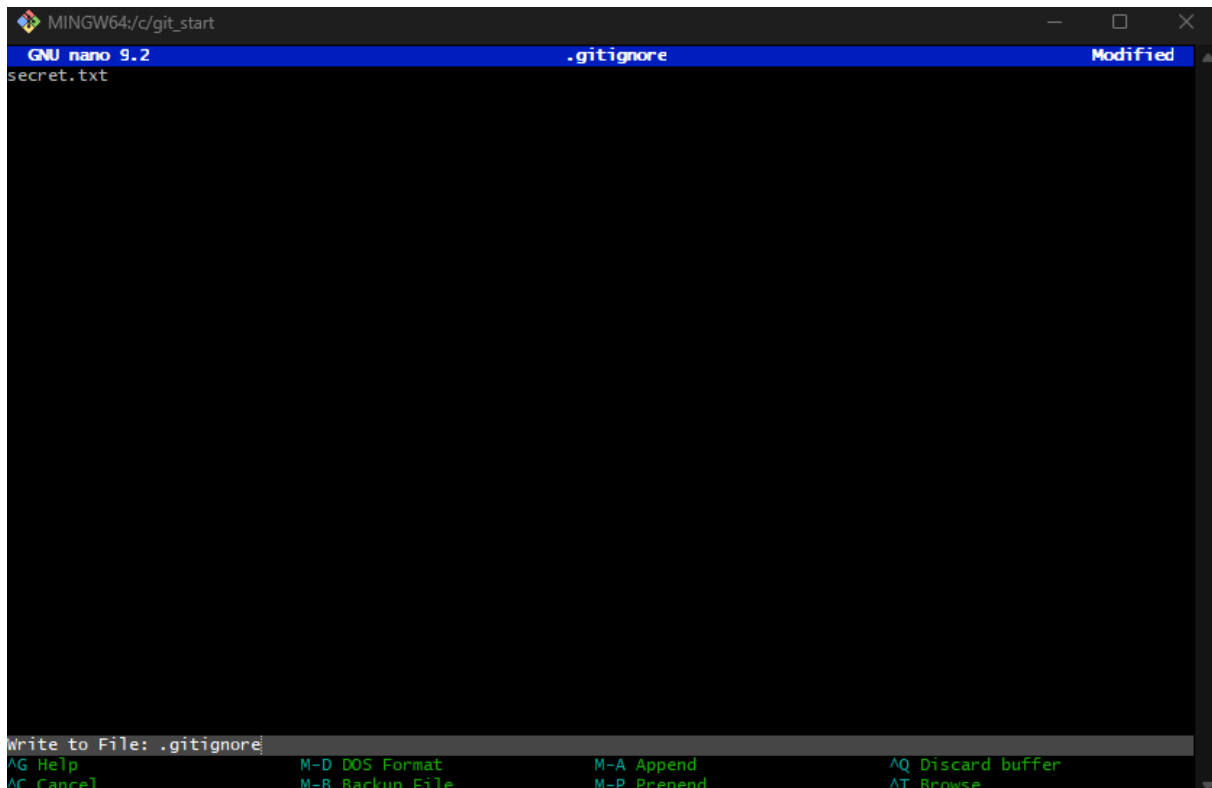
1. **Shift + I** → 아래에 *-insert-* 표시가 뜨면 내용 수정
2. 수정 후 **esc** → 원래 모드로 복귀
3. **esc** 누른 채 **Shift + :** → **:wq** 입력 (w 저장, q 종료)

```
$ git commit -m "vim을 열지 않고 바로 커밋 메시지를 쓰겠다."
```

*gitignore

: 깃 문서 중 수정 이력에서 제외하고 싶은 문서가 있다면 깃 설정 파일인 .gitignore를 사용한다.

\$ touch .gitignore 로 따로 만들어서 사용해야 한다.

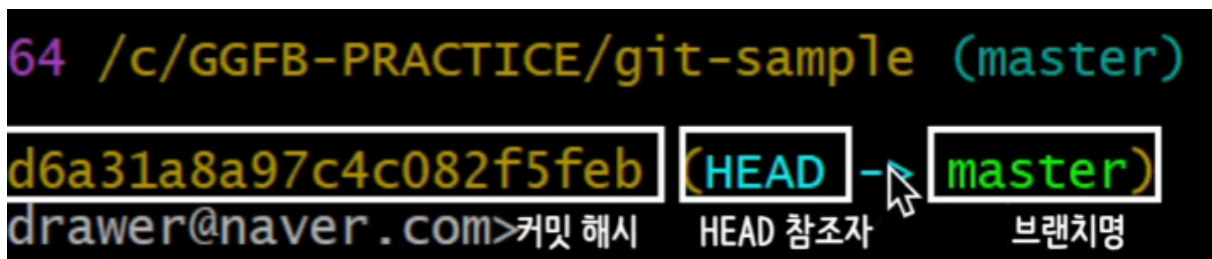


gitignore 작성 (nano)

```
$ nano .gitignore
```

- gitignore 안에 숨길 파일을 적기 위한 인터페이스
1. 숨길 파일 이름을 적고
 2. **Ctrl + X**를 누르면 저장할 것인지 물어오고
 3. yes 이후 그림과 같은 상태에서 enter하면 저장된다.

*커밋 이력



- 커밋 해시 : 커밋 기록에 대한 고유 식별자

- **브랜치명** : 저장소의 내용이 추후 복사가 되면 복사본과 구분하기 위해 사용하는 저장소의 이름
- **HEAD** : 현재 내가 작업 중인 곳을 표시하는 의미. HEAD는 현재 작업 중인 master라는 저장소를 가리키고 있는 손가락 같은 것

*git log 옵션 & 커밋 이동 명령어

```
$ git log -p
```

- 커밋의 변경사항까지 함께 출력하는 옵션, q를 눌러 나갈 수 있다.

```
$ git log --oneline
```

- 각 커밋을 요약해 한 줄 씩 출력하는 옵션

```
$ git checkout 커밋해시
```

- 문서 내용을 특정 커밋의 내용으로 되돌리는 git 명령어

```
$ git reflog
```

- HEAD 포인터의 참조 이력을 출력하는 git 명령어

*실수 했을 때

- 커밋할 생각이 없는데 git add를 해버렸다

\$ git reset "파일명" 명령어로 staging area에 있던 파일을 working directory로 되돌릴 수 있다.

- 실수로 git commit을 해버렸다면?

\$ git reset 명령어와 커밋 해시를 함께 입력하면 커밋 이력 되돌리기가 가능하다. —hard, —mixed, —soft의 세 가지 옵션 중 하나를 선택할 수 있다.

	working directory	staging area	repository
—soft	현 상태 유지	현 상태 유지	커밋 이동
—mixed	현 상태 유지	스테이징 초기화	커밋 이동

—hard	이동한 커밋의 상태로 변경	스테이징 초기화	커밋 이동
-------	----------------	----------	-------

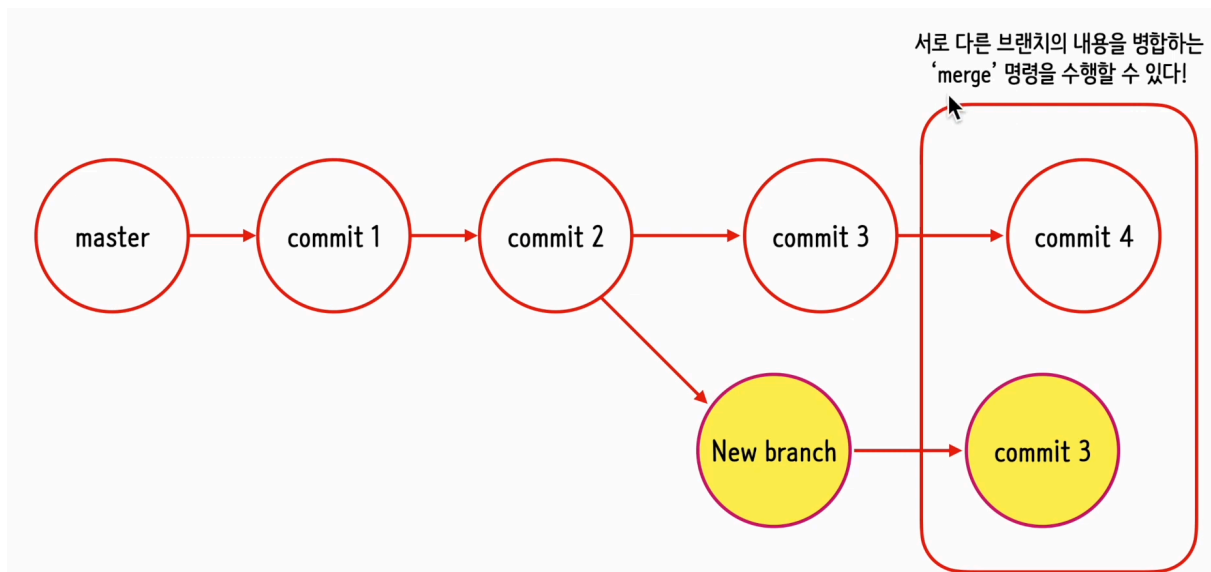
- 불필요한 수정 사항을 없애고 싶은데, 이조차도 기록으로 남기고 싶다

\$ git revert 명령을 사용해 특정 커밋의 수정 사항을 되돌리되, '되돌렸다'는 사실을 이력으로 남기는 명령이다. \$ git revert 231adfv - 231adfv 커밋을 없애거나 되돌리는 게 아닌 해당 커밋에서 기록된 수정 사항을 되돌리는 것 - vim 열고 Shift + i 로 insert 상태에서 메시지 작성 후 esc + : , wq로 나오기

*브랜치 관리하기

: 코드를 통째로 복사하고 원래 코드와 상관없이 독립적인 개발을 진행할 수 있도록 하는 것이 목적.

기존 저장소에서 분기된 저장소의 '복사본'이다



*깃 브랜치 관리하기 명령어

```
$ git branch
```

- 현재 브랜치 목록을 볼 수 있는 깃 명령어

```
$ git branch 브랜치이름
```

- 새로운 브랜치를 생성하는 깃 명령어

```
$ git checkout 브랜치이름
```

- 작업 중인 브랜치를 변경하는 git 명령어

```
$ git merge 브랜치이름
```

- 현재 브랜치에 다른 브랜치의 내용을 병합하는 git 명령어

```
$ git branch -d 브랜치이름
```

- 브랜치 삭제

```
user@DESKTOP-SOQSIDE MINGW64 /c/git-branch-test (master)
$ git log --oneline
4c5def3 (HEAD -> master, dev) third commit on dev
6b959bf second commit
39963b4 first commit

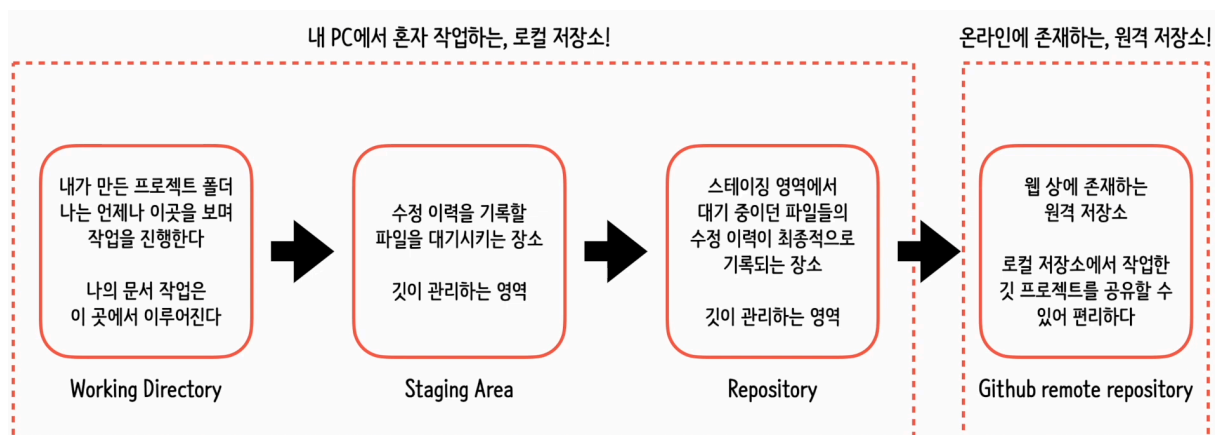
user@DESKTOP-SOQSIDE MINGW64 /c/git-branch-test (master)
$ ls
content.txt test.txt

user@DESKTOP-SOQSIDE MINGW64 /c/git-branch-test (master)
$ git branch -d dev
Deleted branch dev (was 4c5def3).

user@DESKTOP-SOQSIDE MINGW64 /c/git-branch-test (master)
$ git branch
* master

user@DESKTOP-SOQSIDE MINGW64 /c/git-branch-test (master)
$ ...
```

**깃허브!



*원격 저장소 이용 명령어

```
$ git remote -v
```

- 현재 깃 프로젝트에 등록된 원격 저장소를 확인하는 깃 명령어

```
$ git remote add 원격저장소이름 원격저장소주소
```

- 현재 깃 프로젝트에 원격 저장소를 등록하고, 여기에 이름을 붙이는 깃 명령어

```
$ git push
```

- 로컬 저장소의 내용을 원격 저장소에 공유할 때 사용하는 깃 명령어

```
$ git pull
```

- 원격 저장소의 내용을 로컬 저장소로 가져와 자동 병합하는 깃 명령어

- 새롭게 깃허브를 연결할 때

```
er@DESKTOP-SOQSIDE MINGW64  
git remote add origin http
```

*보통 관례 상 origin이라고 이름을 붙인다고 한다



처음 git push를 할 때는 정확한 경로 설정이 필요해 에러가 날 수 있다. `$ git push -u origin master`로 origin이라는 원격 저장소에 master라는 브랜치를 명시하겠다는 명령어를 사용한 후에 push 할 수 있다.

- 깃허브에서 만든 자료를 가지고 올 때

```
user@DESKTOP-SOQSIDE MINGW64 /c/workspace (master)
$ cd ..

user@DESKTOP-SOQSIDE MINGW64 /c
$ git clone https://github.com/dauil727/AI-Service.git
Cloning into 'AI-Service'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.

user@DESKTOP-SOQSIDE MINGW64 /c
$ ..
```

가지고 올 부분의 상위 폴더로 간 이후 주소를 복제하면 깃허브의 폴더가 다운로드 된다.